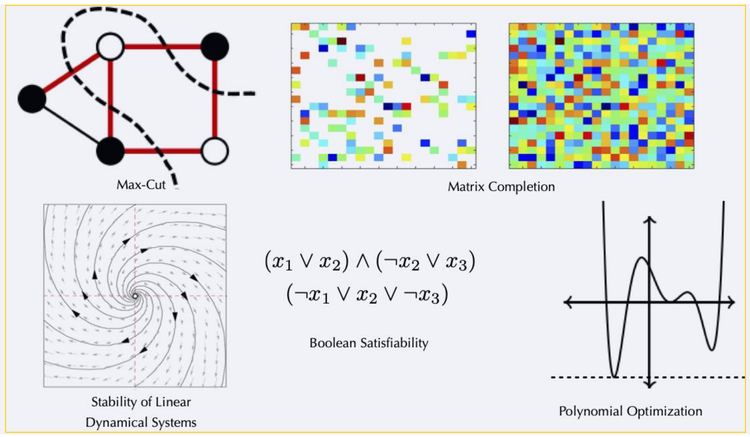
Fast, Scalable, Warm-Start Semidefinite Programming with Spectral Bundling and Sketching

Rico Angell[1] and Andrew McCallum[1]
[1]University of Massachusetts Amherst

Motivation

- Semidefinite programming (SDP) is an important tool for many applications

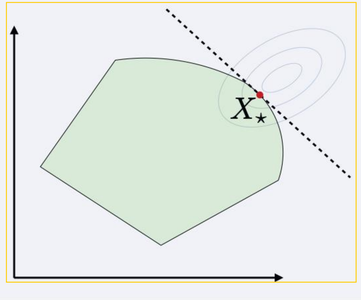


- Many existing methods for solving or approximately solving SDPs
- Need for a method that is fast, scalable, and provably correct for massive SDPs
- Need a method where hyperparameters can modulate memory, accuracy, speed, and scalability of solving general SDPs

Problem Setup

$$\max_{X \succeq 0} \langle C, X \rangle \quad \text{s.t.} \quad \mathcal{A}_{\mathcal{I}'} X = b_{\mathcal{I}'}, \quad \mathcal{A}_{\mathcal{I}} X \leq b_{\mathcal{I}}$$

$\mathcal{A} : \mathbb{S}^n \to \mathbb{R}^m$ linear operator



- Want to solve general SDPs with both equality and inequality constraints
- Want to have theoretical guarantees about convergence to optimal solution
- Assume cost matrix C is sparse and a upper bound $\alpha \geq \mathrm{tr}(X_\star)$ exists
- Assume the number of constraints is small compared to the problem size
- Want to be able to solve SDPs where X will not fit in memory
- Want solver to be able to take advantage of a warm-start initialization

Unified Spectral Bundling

- Reformulated SDP as a the following penalized dual objective:

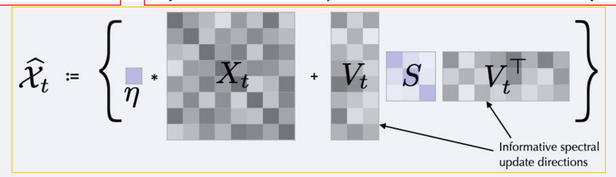
$$f(y) := \alpha\,[\lambda_{\max}(C - \mathcal{A}^* y)]_+ + \langle b, y \rangle + \iota_{\mathcal{Y}}(y) \qquad \mathcal{Y} \geq 0$$
$$= \sup_{(X,\nu) \in \mathcal{X} \times \mathbb{N}} \langle C - \mathcal{A}^* y, X \rangle + \langle b - \nu, y \rangle$$

Dual slack variable for inequality constraint on y

- Solve the SDP by minimizing f using a proximal bundle method:

$$\tilde{y}_{t+1} \leftarrow \arg\min_{y \in \mathbb{R}^m} \hat{f}_t(y) + \frac{\rho}{2}\|y - y_t\|^2$$

where the model $\hat{f}_t$ is parameterized by the low dimensional subspace of $\mathcal{X}$:



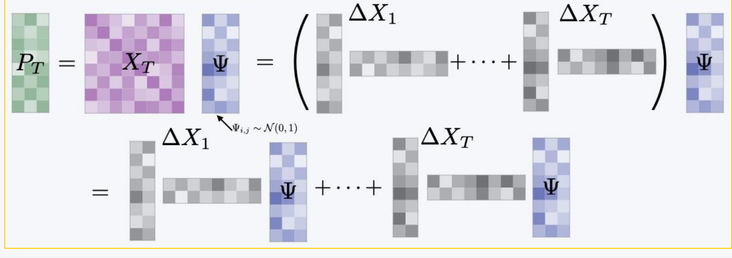
- The size of V determines the rank k of each update, specified by the user
- Only update $y_{t+1} \leftarrow \tilde{y}_{t+1}$ if $\beta(f(y_t) - \hat{f}_t(\tilde{y}_{t+1})) \leq f(y_t) - f(\tilde{y}_{t+1})$
- Always use $\tilde{y}_{t+1}$ to create $\hat{\mathcal{X}}_{t+1}$, V_{t+1} is the top-k eigenvectors of $C - \mathcal{A}^* \tilde{y}_{t+1}$
- Analytical solution to the proximal bundle method lead to the following updates

$$\tilde{y}_{t+1} = y_t - \frac{1}{\rho}(b - \nu_{t+1} - \mathcal{A}X_{t+1})$$
$$(X_{t+1}, \nu_{t+1}) \in \arg\max_{(X,\nu) \in \hat{\mathcal{X}}_t \times \mathbb{N}} \langle C, X \rangle + \langle b - \nu - \mathcal{A}X, y_t \rangle - \frac{1}{2\rho}\|b - \nu - \mathcal{A}X\|^2$$

- $f(y_t) - \inf f \leq \varepsilon$ after $t \geq O(1/\varepsilon)$ iterations, $O(\cdot)$ suppresses dependence on n, m

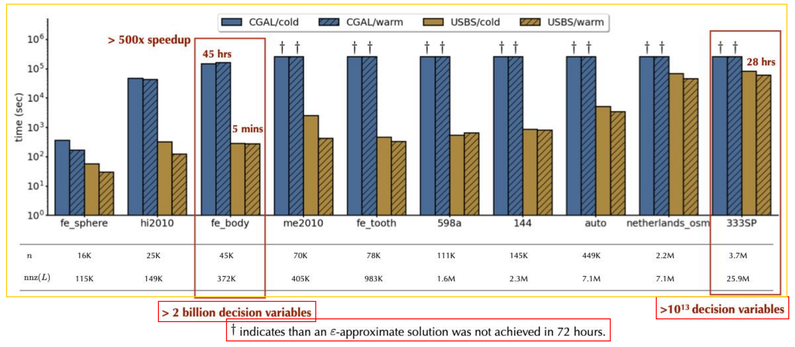
Scaling with Matrix Sketching

- Only need to keep track of projections of X such as $\langle C, X \rangle$ and $\mathcal{A}X$
- All of the memoization can be done efficiently due to linearity of operations
- Apply low-rank matrix sketching techniques to enable scaling to problems where X cannot fit in memory — track projection P instead of X

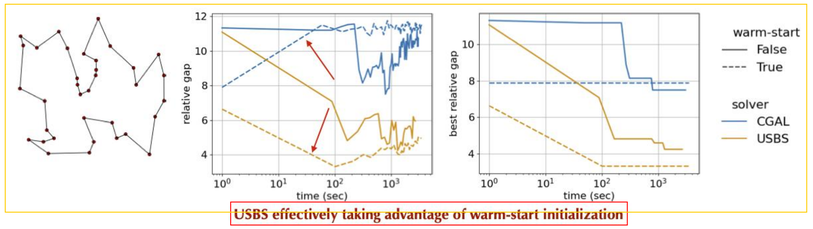


Results

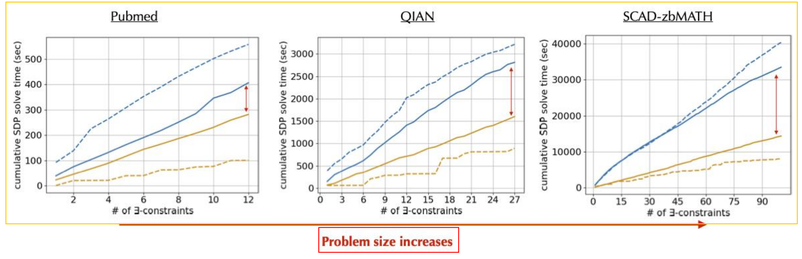
Max-Cut



Traveling Salesman Problem



Clustering with ∃-constraints



UMassAmherst
Manning College of Information & Computer Sciences